Syamasudha Veeragandham

23BCT0122_crashgaurd_piot_report

Student Project

Document Details

Submission ID
trn:oid:::3618:155969675

Submission Date
Oct 26, 2026, 8:37 PM GMT+5:30

Download Date
Oct 26, 2026, 8:43 PM GMT+5:30

File Name
23BCT0122_crashgaurd_piot_report.pdf

File Size
717.7 KB

23 Pages

13,573 Words

65,823 Characters

44% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

68 AI-generated 44%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

CrashGuard: Simulation-Based Evaluation of an IoT Crash Detection and Emergency Alert System for Two-Wheeler Riders

Adheesh Garg[1], Ayan Gatani[1], Aastha Gupta[1],
Syamasudha Veeragandham[2,*]

[1,2]School of Computer Science and Engineering,
Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

adheesh.garg2023@vitstudent.ac.in
ayan.gatani2023@vitstudent.ac.in
aastha.gupta2023@vitstudent.ac.in
[*]Corresponding author: syamasudha.v@vit.ac.in

Abstract

This paper describes CrashGuard, a low-cost device that detects motorcycle crashes and sends an emergency alert, and evaluates it by simulation. An ESP32 microcontroller samples a six-axis inertial sensor on the handlebar at 100 Hz, and a small finite-state machine decides whether a crash has occurred. A crash is confirmed only if four things happen in order: the motorcycle has been ridden, there is an impact or a sudden unloading, a strong rotation follows within a short time, and the motorcycle then lies tilted and still for two seconds. On confirmation the device switches on a light and a buzzer, and the rider has 15 seconds to cancel before a single event is sent to a cloud service. No physical prototype was built. All results come from generated motion data, from a public motorcycle dataset, and from the unchanged firmware run against simulated peripherals.

On 270 freshly generated events, CrashGuard rejected all 210 non-crash events and detected all 30 high-side crashes, but it missed 17 of the 30 low-side crashes, so its overall sensitivity is 71.7% (F1 = 0.835). A rule that follows the staged structure of the motorcycle fall detector of Boubezoul et al. (2013), run on the same events, detected more crashes (86.7%) but raised 34 false alarms; McNemar's test shows that the two rules differ significantly ($\chi^2 = 13.40$, $p = 0.00025$). Sweeps over the tilt, rotation and rest-hold thresholds, one parameter at a time, indicate that the 2 s rest hold is what suppresses the nuisance alarms, and that a lower rotation gate could recover some of the missed low-side crashes, although the latter still has to be confirmed on separate data. Fourteen firmware tests found three defects: the rest timer keeps running across missing sensor reads, a failed cloud connection blocks the main loop for 5 s, and an alert that falls due during a Wi-Fi outage is never retried. A simple battery model gives 12.64 hours of operation at a 120 mA, 5 V load. Code, data and scripts are released so that every experiment reported here can be repeated.

Keywords: IoT; ESP32; MPU6050; Motorcycles; Detection; Simulation; Alerts; Safety.

1 Introduction

A rider who falls from a motorcycle on an empty road may remain unnoticed for some time. A device on the motorcycle that recognises the crash and calls for help can shorten this delay, but only if it does not raise false alarms so often that the rider switches it off. The task is thus a detection problem with two kinds of error, and the two do not carry the same cost. A missed crash can leave an injured rider without help. A false alarm costs little each time, but frequent false alarms make the device useless. In this paper we describe CrashGuard, a handlebar-mounted prototype built around an ESP32 microcontroller and an MPU-6050 inertial sensor, and we evaluate it. We say at the outset that no physical hardware was assembled; the evaluation uses generated motion data, public motorcycle recordings, and the real firmware run against simulated peripherals. The remainder of this section reviews, one paper at a time, the earlier work that shaped the design.

Boubezoul et al. [1] were among the first to treat fall detection for powered two-wheelers (PTWs) as a problem in its own right. Their main observation is that the lean of a motorcycle in ordinary riding, together with shocks from the road surface, makes a single threshold on acceleration unreliable. They detect a fall in stages: the start of the fall, the growth of the lean angle, and a period of rest after the fall. CrashGuard keeps this staged view. Because it has to run on an inexpensive handlebar unit, however, the stages are written as a small finite-state machine, with a ride-qualification step in front and a cancellation window behind. We compare our detector with a rule which follows the structure of their algorithm, and with two simpler rules, on exactly the same six-axis data.

A few years later the same group published recordings of real PTW falls and critical manoeuvres [2]. Data of this kind are rare, as crashing a motorcycle on purpose is expensive and dangerous. The falls are the positive examples, and the manoeuvres (hard braking, acceleration in a curve, a badly degraded track, and so on) are negative examples which test whether a detector can stay quiet. We take four falls and seven manoeuvres from this set and use them for a limited purpose: we check whether the signals cross the acceleration and rotation gates of CrashGuard, that is, we test the scale of our thresholds and not the full decision. All four falls cross both gates. The unit conversion, filtering and choice of time windows are published with the code.

Gelmini et al. [3] developed a crash detection algorithm for two-wheeled vehicles and tested it on experimental riding data. Their work matters to us chiefly as a lesson about coverage. Ordinary riding produces a wide variety of signals, and a detector which has been shown only a few convenient crash examples tells us little about its behaviour on the road. Our simulated data therefore contain braking, sharp turns, potholes, speed breakers, an impact followed by recovery, and a parked motorcycle tipping over, as well as two kinds of crash. Low-side and high-side crashes are kept as separate classes. The separation turns out to matter: in the fresh data every high-side crash is detected, and all 17 misses are low-side crashes.

In the literature on human falls, Sucerquia et al. [4] introduced SisFall, a dataset recorded with a device worn at the waist. Its signals come from the human body rather than a handlebar, so we do not use them. What we borrow is its design: it contains many everyday activities which are not falls, so a detector evaluated on it has to show that it stays quiet as well as that it fires. We follow the same principle. Seven of our nine scenario classes are negative, and the class counts are given with every result. On the fresh events CrashGuard rejects all 210 negatives, although this holds, of course, only within our signal model.

Casilari et al. [5] compared the public datasets for wearable fall detection and found large differences in sensor position, sampling rate, recorded activities and recording protocol. A sensitivity of, say, 95% measured on one dataset may say little about another. For a new study, the practical consequence is that the data must be described in enough detail for a reader to judge what the numbers mean. Accordingly, we report the channel units, the sampling rate, the random seeds, the noise and bias levels, the clipping limits and the kinematic transformation used by our generator. Events are never split across the development, validation and test subsets, and a second random stream is used to check that the results repeat.

In a later study, Casilari et al. [6] trained convolutional networks on several public fall datasets, and showed that the outcome depends strongly on which repository is used. This result made us cautious about what our own second dataset can show. A new random seed gives new events, but they come from the same generator, the same mounting position and the same model of the motorcycle. We call the result a fresh-seed evaluation and claim no more for it. The public PTW recordings give evidence of a different kind, about the physical scale of the signals, and the two are reported in separate tables. A stronger test, with whole vehicles or riding sessions held out, is left for future work.

Musci et al. [7] used recurrent neural networks (LSTMs) for online fall detection on an annotated version of SisFall. Their model reads a sequence of samples rather than single samples, and we agree that the order of events in time carries most of the information. We simply encode it differently. Instead of a learned model we use explicit timing rules: a correlation window between impact and rotation, a verification timeout, and a hold time for the rest condition. One advantage of explicit rules is that the same C++ code runs in the offline replay and in the firmware, so the two cannot disagree. Another is that, when a fault is injected, the transition at which the behaviour went wrong can be read directly from the log.

Yu et al. [8] published KFall, a large dataset with temporal labels, intended for pre-impact fall detection. A pre-impact detector, for example one which triggers a wearable airbag, must act within a fraction of a second. Our device acts after the event and has a few seconds to spare. The two problems therefore call for different timing measures, and it matters where a delay measurement begins and ends. We measure the time from the first candidate sample to confirmation separately from the 15 s cancellation window which follows it. For the fresh events the median of the former is 2.33 s. The firmware tests check that the deadline at the end of the window is met to the millisecond.

Rodegast et al. [9] trained machine-learning models for motorcycle collision detection on simulated accidents and simulated normal riding. Their target is a passive safety system, which must fire quickly and must not fire by mistake. Our target is a warning which the rider can cancel, so a slower decision is acceptable, but nuisance alarms remain costly. Like them, we rely on simulation. Our generated data were produced after the detector rules had been fixed, and every rule we compare receives exactly the same events, so the trade-off between missed crashes and false alarms can be read directly from the confusion counts.

Leoni et al. [10] proposed an automatic eCall method for motorcycles based on vehicle dynamics. They distinguish high-side crashes, with fast rotation of the vehicle, from low-side crashes, in which the motorcycle slides out more slowly. This distinction is central to our results. A threshold tuned to a fast, violent high-side crash may simply miss a slow slide. In our data CrashGuard rejects every non-crash event, but its low-side recall on the fresh set is only 43.3%, and we regard this as the main weakness of the detector. We report it separately so that the overall accuracy of 93.7% is not read as uniform protection.

Tian et al. [11] used a multi-head attention transformer for fall detection with a sensor worn on the shin. They report a test sensitivity of 90.5% and an overall accuracy of 93.1%, and they discuss how class imbalance affects numbers of this kind. Their sensor sits on a human leg and ours on a handlebar, so the figures are not comparable, and we make no attempt to compare them. We cite the work as recent evidence of what learned sequence models can do. From it we also take the habit of reporting the class counts, F1 and sensitivity next to accuracy; with 210 negatives and only 60 positives, accuracy alone would flatter our detector.

Finally, Pradhan et al. [12] combined a dual-stream transformer with Kalman-filter fusion of inertial signals, and report an F1-score of 91.10% on SmartFallMM in a subject-wise evaluation. Two of its ideas seem worth adopting in later work, namely the fusion of complementary inertial signals and the holding out of whole groups (for us, vehicles or riding sessions) during evaluation. At present CrashGuard uses three hand-designed features and fixed rules. Its contribution lies elsewhere, in a light decision sequence whose every state can be observed and tested.

1.1 Research Gaps and Demonstrated Improvements

Identified Research Gaps

Taken together, the work above leaves three gaps that matter for a small device of the kind we build.

Gap 1: Thresholds on acceleration alone. A fixed threshold on the acceleration magnitude cannot both ignore hard road shocks (potholes, rumble strips, speed breakers) and detect slow, low-energy slides.

Gap 2: Overall accuracy hides missed low-side crashes. When most test events are negative, a high overall accuracy can coexist with poor detection of one type of crash, typically the low-side slide in which the deceleration is gentle.

Gap 3: The alert chain is rarely tested. Most studies stop at classifier metrics. They do not test what the device does when the sensor fails, when the rider cancels, when the cloud service does not answer, or when the battery runs down.

Demonstrated Engineering Contributions

The staged decision is due to Boubezoul et al. [1]. Our contributions concern its use in a small handlebar device, and its evaluation.

1. **Ride qualification.** A leaky-bucket filter of recent activity, so that a parked motorcycle which tips over, or a knock while standing, is not taken for a crash.
2. **Local warning and cancellation.** A light and a buzzer, followed by a fixed 15 s window in which the rider can cancel, before anything is sent.
3. **Comparison on identical data.** CrashGuard is compared with a rule following the structure of Boubezoul et al. [1] and with two simpler rules on the same events, with paired McNemar tests.
4. **Threshold sweeps.** The tilt, rotation and rest-hold thresholds are varied one at a time over 29 settings, to see which of them controls the missed low-side crashes and which controls the false alarms.
5. **Firmware fault tests.** The unchanged firmware is run against simulated peripherals in 14 scenarios, which exposed three defects: in the rest timer, in a blocking network call, and in the missing retry after a network outage.

We repeat that no hardware was built or measured; every finding comes from simulation, from models of virtual peripherals, or from public recordings.

1.2 Research Objectives

1. To implement a 100 Hz six-axis state machine which detects crash-like events on a motorcycle handlebar, with a local warning and cancellation by the rider, and to evaluate it.
2. To compare CrashGuard with a rule following Boubezoul et al. [1] and with two simpler rules on the primary and the fresh-seed event sets, reporting confusion counts, sensitivity, specificity, F1, delay and paired McNemar tests.
3. To find out, by sweeping the rest-tilt, rotation and rest-hold thresholds, how each of them affects low-side recall and false alarms.
4. To run the firmware with controlled virtual peripherals under sensor, calibration, timing and communication faults, and to record the expected and the actual behaviour.
5. To publish the data, plots, circuit and architecture diagrams and power model, together with a plain account of the limitations.

2 Related Work

This section regroups the same sources by topic and states what each one does; the introduction has already explained how each affects our design.

Motorcycle fall and crash detection. Boubezoul et al. [1] gave a simple fall detection algorithm for PTWs, in which a fall is recognised in stages from the motion of the vehicle. The same group later released recordings of PTW falls and critical manoeuvres [2], which we use for an external check of signal scale. Gelmini et al. [3] studied crash detection for two-wheeled vehicles on experimental riding data and stressed how varied ordinary riding is. Rodegast et al. [9] trained classifiers on simulated motorcycle collisions for fast activation of passive safety systems. Leoni et al. [10] gave a dynamics-based eCall method, in which fast high-side motion and slower side slips are treated differently.

Human fall detection and its datasets. SisFall [4] contains waist-worn recordings of falls and of activities of daily living. Casilari et al. [5] compared such public datasets and showed how much they differ in sensor position, sampling and protocol, and in a later study [6] they showed that the results of convolutional networks depend on the dataset used. KFall [8] added synchronised temporal labels and benchmark algorithms for pre-impact detection.

Learned sequence models. Musci et al. [7] applied LSTM networks to online fall detection on an annotated version of SisFall. Tian et al. [11] used a multi-head attention transformer for a shin-mounted device and report 90.5% test sensitivity and 93.1% accuracy after class balancing and focal-loss training. Pradhan et al. [12] combined a dual-stream transformer with Kalman-based fusion and report 91.10% F1 in a 21-fold leave-one-subject-out evaluation on SmartFallMM. The mountings, labels and populations in these studies differ from ours, so we use their numbers as context and not as a ranking.

The cloud interface. The Blynk documentation on device templates and on logging events [13, 14] defines the device identity, the event code and the firmware call which we use to report an incident. Although this is not detection research, it fixes the boundary between the controller and the cloud, and our firmware tests record what happens at this boundary when an event is sent, when it is skipped, and when the connection times out.

Table 1 collects these sources. For each it gives the sensing domain or method and states which part of our design or evaluation it informs. The rows describe how the sources relate to our work; they do not rank methods on a common dataset.

Table 1: Prior work on motorcycle crashes, human falls, datasets and the IoT interface, and its role in CrashGuard. The first column names the source, the second gives its sensing domain or method, and the third says which design or evaluation decision it informs. The public motorcycle data serve as an external signal reference; the human-fall studies inform the method of evaluation but use other mounting positions and labels.

Source	Sensing domain / method	Relevance to this work
Boubezoul et al. (2013) [1]	Motorcycle fall algorithm	Source of the staged decision. Our rules expose the candidate, rotation and rest states, so that each can be replayed.
Boubezoul et al. (2019) [2]	Public PTW recordings	Four controlled falls and seven critical manoeuvres. We keep their labels and check the acceleration and rotation gates separately.
Gelmini et al. (2021) [3]	Motorcycle crash detection	Reason for covering many vehicle behaviours; braking, turning, road shocks and recovery are explicit negative classes in our data.
SisFall (2017) [4]	Waist inertial dataset	Shows the value of everyday non-fall examples. We carry over the principle, not the human-body labels.
Casilari et al. (2017) [5]	Dataset analysis	Reason for reporting sensor location, sampling rate, units, event composition and the split procedure.
Musci et al. (2018) [7]	Online LSTM detection	A learned temporal model; CrashGuard writes the same kind of timing as explicit correlation and rest rules.
Tian et al. (2026) [11]	Shin-mounted transformer	Recent attention-based work on human falls, sensitive to class balance. The mounting differs from our handlebar.
KFall (2021) [8]	Pre-impact timing dataset	Separates pre-impact timing from post-event detection. We report the confirmation delay apart from the cancellation window.
Casilari et al. (2020) [6]	CNNs on several datasets	Reason for keeping the fresh random stream apart from the public PTW data, which come from another acquisition domain.
Rodegast et al. (2024) [9]	Simulated collisions, learned models	Fast activation of protective systems; our application is a warning which can be cancelled.
Leoni et al. (2025) [10]	Dynamics-based motorcycle eCall	Separates fast high-side motion from slower side slips, which is why we report low-side misses on their own.
Pradhan et al. (2026) [12]	Dual-stream inertial fusion	Kalman fusion and subject-wise evaluation; suggests vehicle-wise or session-wise evaluation for us.
Blynk documentation [14]	Device event interface	Defines the event code and description. We instrument the submission and do not equate it with a notification received on a phone.

Summary of Gaps Addressed

In short, three gaps motivate both the design and the experiments.

- **Choice of threshold.** A high acceleration threshold misses slow, low-energy falls; a low one fires on ordinary road unevenness.
- **Scope of the evaluation.** Accuracy is not enough for a safety device. Calibration, cancellation by the rider, alarm deadlines and the behaviour of the network all affect what happens after a real crash.
- **Openness.** Without shared detector code, fixed event datasets and fault traces, other people cannot check the results or find the defects.

Problem Statement

We are given a 100 Hz stream of three-axis acceleration and three-axis angular rate from a motorcycle handlebar. The task is to decide whether, during active riding, the stream contains an impact or an unloading, a correlated rotation, and a sustained tilted rest, in this order. When this is confirmed, the device switches on its local outputs and allows 15 seconds for cancellation before it submits a single IoT event. The evaluation must report false alarms, missed falls, detection delay and the behaviour under faults, by experiments which others can repeat.

3 Materials and Methods

3.1 Experimental platform and materials

The design targets an ESP32-WROOM-32 development board, a GY-521 MPU-6050 module, a red LED with a 220 Ω resistor, a 5 V passive piezo driven through a transistor, and an active-low push-button. Power comes from a protected 2600 mAh 18650 cell, a TP4056/DW01A charger and protection module, and an MT3608 boost stage giving 5 V. The firmware sets the sensor to I^2C address `0x68`, a 400 kHz bus, the 44 Hz digital low-pass setting, 100 Hz output, and the ranges $\pm 16g$ for acceleration and $\pm 2000°/s$ for angular rate [15]. As stated in Section 1, no hardware was assembled or measured; the schematic, the interface calculations and the discharge curves are design and simulation results.

The experiments use the real detector header and the real firmware sketch [16]. A native C++ replay runs the detector on the generated motion data. A separate harness compiles the unchanged sketch together with deterministic stand-ins (test doubles) for the Arduino core, the Wire library, Wi-Fi and a Blynk-compatible service. Deadlines follow a virtual clock, and signed register values are fed through the raw I^2C read path. The harness records the digital pin states, the tone frequency, the serial messages, the event payloads and the connection delays. In what follows, “the cloud” always refers to this simulated service.

Table 2 lists how each sensing, warning and input function is connected to the controller. Because the harness observes exactly these pins, the table also serves as the link between circuit and software.

Table 2: ESP32 interface allocation. GPIO is a general-purpose input/output pin, SDA and SCL are the I^2C data and clock lines, and ADC1 is the first analogue-to-digital converter. All logic is referred to a common ground. The outputs listed are the firmware commands observed in the harness; the battery entry gives the divider used to recover the cell voltage from the ADC reading.

Element	ESP32 interface	Configuration	Function
MPU-6050	GPIO21 / GPIO22	SDA / SCL	Acceleration and angular-rate acquisition.
LED	GPIO26	Active high	Visible crash warning.
Piezo driver	GPIO25	2400 Hz tone	Audible warning.
Cancel button	GPIO27	Internal pull-up	Cancel or acknowledge an alert.
Battery divider	GPIO34 / ADC1	100 kΩ / 100 kΩ	Cell voltage measurement.
Wi-Fi	Station interface	Phone hotspot	Blynk SSL event path.

3.2 System architecture

Figure 1 follows the data from the inertial sensor, through the decision made on the ESP32, to the local warning and then to the remote event. The power supply and the rider’s button are drawn separately, since they support the decision path but are not part of it.

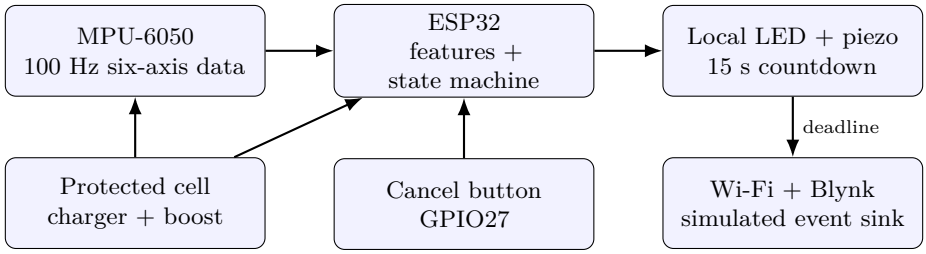


Figure 1: Architecture of CrashGuard: acquisition, local decision, cancellation by the rider, and the remote event. In the harness the event sink is replaced by an instrumented model of the service.

3.3 Circuit diagram and virtual-hardware image

Figure 2 gives the circuit at connection level: power conditioning, the sensor, the battery measurement and the warning outputs, with common nets marked by labels. The component values are the ones used in the DC interface calculation below, and the drawing also shows how the transistor drives the piezo.

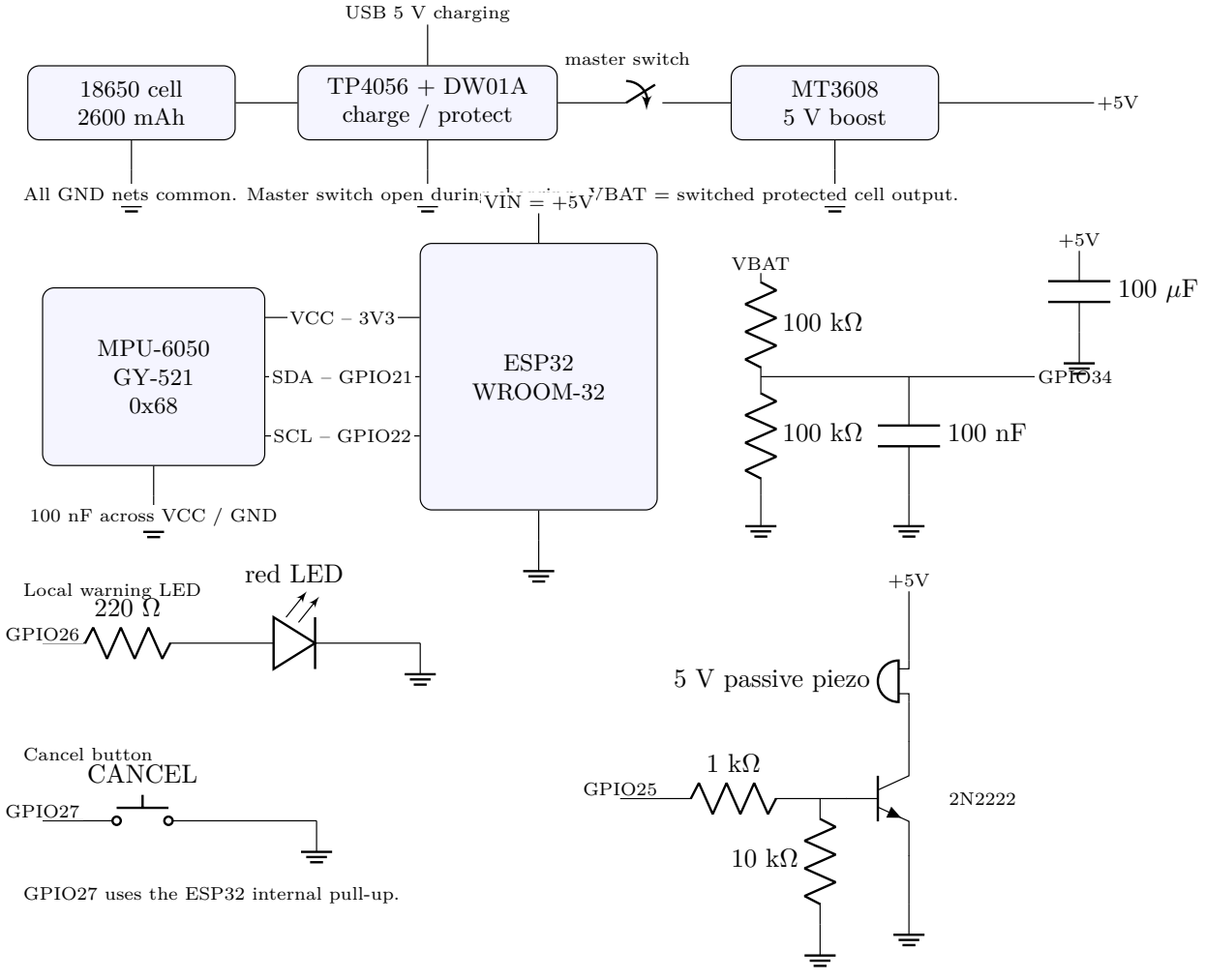


Figure 2: Connection-level circuit: power, sensing, battery divider, LED, transistor-driven piezo and cancel input. The master switch is open during charging, as the charger module does not share the load.

The GY-521 module runs at 3.3 V for both power and logic. GPIO25 drives the piezo through a 2N2222 transistor, with a 1 kΩ base resistor and a 10 kΩ pull-down. The battery divider has a 100 nF capacitor to ground, and the 5 V rail is decoupled by 100 µF. We computed the DC operating points for 39 combinations of cell voltage (3.0 to 4.2 V) and LED forward drop (1.8, 2.0 and 2.2 V). The ADC input then ranges from

1.50 to 2.10 V, the divider draws 15 to 21 μA, and the LED current is between 5.0 and 6.82 mA. These values are calculated rather than measured, and they are independent of the battery experiment in Section 4.9.

Figure 3 captures one particular executed state rather than an arbitrary arrangement of parts. It was taken at the moment of confirmation, 3.610 s into the run, and its annotations give the LED level, the tone frequency, the countdown and the converted battery reading at that moment.

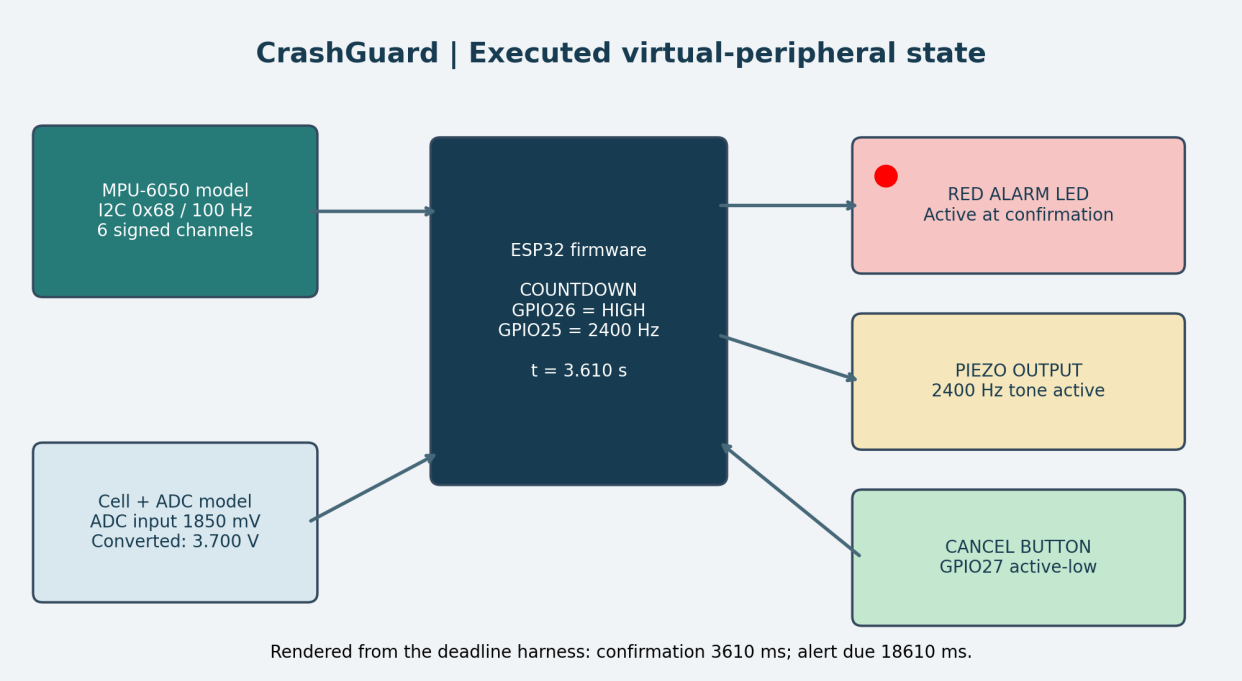


Figure 3: Virtual hardware at the confirmation instant, 3.610 s. The board and peripherals show the state recorded by the harness: LED on, 2400 Hz tone, countdown running.

3.4 Dataset generation and preprocessing

The primary simulation has 675 events, each ten seconds long at 100 Hz, so 675,000 six-axis samples in all. There are nine classes of 75 events. Normal riding, potholes, speed breakers, hard braking, sharp turning, a parked tip-over, and an impact followed by recovery are negative classes; low-side and high-side crashes are positive. This gives 150 positive and 525 negative events. The generator is fixed by the seed 20260828. A second seed, 20260911, gives a further 30 events per class, that is 270 fresh events and 270,000 samples.

Each event is built from roll, pitch and yaw profiles, the gravity vector in the body frame, pulses for the manoeuvre or impact, and noise, bias and clipping. Gravity is expressed in the body frame of the sensor as

$$\widehat{\mathbf{g}}_b = \begin{bmatrix} -\sin\theta \\ \sin\phi\cos\theta \\ \cos\phi\cos\theta \end{bmatrix} \in \mathbb{R}^{3\times 1}, \qquad \mathbf{g}_b = g_0\widehat{\mathbf{g}}_b, \qquad \|\widehat{\mathbf{g}}_b\|_2 = 1, \tag{1}$$

where ϕ and θ are the roll and pitch angles in radians, and the three entries are ordered along the x, y and z axes of the sensor (the subscript b stands for body frame). The hat marks a unit vector, so $\widehat{\mathbf{g}}_b$ is dimensionless and its entries can be read directly in units of g. Multiplying by standard gravity, $g_0 = 9.80665$ m/s^2, gives the physical vector $\mathbf{g}_b$ in m/s^2; g_0 is a constant and not a fitted parameter. For a level sensor, $\phi = \theta = 0$, the vector is $[0, 0, 1]^{\mathsf{T}}$. A positive roll gives a positive y component and a positive pitch a negative x component. Yaw does not appear, since rotation about the vertical does not change the direction of gravity.

Linear pulses representing the road surface, the rider's inputs and the impacts are then added to this gravity vector. To produce the gyroscope signals, the generator does not simply differentiate the Euler angles. It converts the angle rates $\dot{\boldsymbol{\Theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^{\mathsf{T}}$ into body-frame angular velocities $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^{\mathsf{T}}$ with the standard

Tait–Bryan Z-Y-X (yaw–pitch–roll) relation

$$\begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \mathbf{E}(\phi, \theta) \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \sin\phi\cos\theta \\ 0 & -\sin\phi & \cos\phi\cos\theta \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}. \tag{2}$$

Here ψ is the yaw angle and $\mathbf{E}(\phi, \theta)$ is the 3×3 matrix which depends only on roll and pitch. When the motorcycle rolls and yaws at the same time, as in a high-side crash, the matrix mixes the angle rates across the three gyroscope axes, which is what a real body-fixed gyroscope would see. At zero roll and pitch the matrix is the identity, and the body rates are simply the angle rates.

The time of the event is drawn uniformly from 3.8 to 5.2 s. The acceleration bias has standard deviation 0.018 g and the sample noise 0.022 g; for angular rate the corresponding values are 0.8 and 1.5 °/s. Samples are clipped to the full-scale ranges of the sensor. The final roll angle lies between 48 and 105° for low-side crashes and between 70 and 135° for high-side crashes. Note that the low-side range deliberately reaches below the 55° tilt gate of the detector, so some low-side crashes cannot be detected by design. Impact pulses are between 3.0 and 11.5 g for high-side crashes and between 0.8 and 5.5 g for low-side crashes, before vector addition and clipping.

For the external signal analysis we load seven critical manoeuvres and four falls from the published PTW supplement [2]. We keep the finite numeric rows, filter the six channels against aliasing and resample them from 1 kHz to 100 Hz (polyphase resampling with a Kaiser window), convert acceleration from m/s^2 to g by dividing by 9.80665, and keep angular rate in degrees per second. The four fall windows are centred at 40.132, 34.288, 35.876 and 43.486 s, each extending ±1.5 s. The manoeuvres are used whole, as negative examples. These short windows test individual gates only; they are not a test of the full decision with rest and countdown.

3.5 Data split and evaluation protocol

Within each scenario the primary events are split by the order in which they were generated: the first 45 go to development, the next 15 to validation and the last 15 to test. This gives 405, 135 and 135 events, that is 60, 20 and 20%. An event is never divided between subsets. The file `results/splits.csv` lists every assignment.

CrashGuard is a fixed set of rules and has no fitted weights. The split is therefore only a retrospective breakdown of a single replay, and the test subset should not be read as a held-out test in the usual sense. The fresh set of 270 events was generated after the detector had been frozen for this study, and no threshold was chosen by looking at it. It is a second random stream from the same family of scenarios. The public PTW recordings are reserved for the gate analysis, and the human-fall datasets are used only as context.

3.6 Features, calibration, and decision equations

At start-up the firmware averages 200 stationary samples. If the norm of the averaged acceleration lies outside 0.85–1.15 g, or the RMS of the gyroscope exceeds 5 °/s, calibration is repeated. Otherwise the averaged acceleration becomes the reference gravity vector, and the three gyroscope means become bias offsets. Let $\mathbf{a} = [a_x, a_y, a_z]^{\mathsf{T}}$ be the acceleration vector, $\mathbf{r} = [r_x, r_y, r_z]^{\mathsf{T}}$ the reference gravity vector, both in units of g, and $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^{\mathsf{T}}$ the angular rates in degrees per second. The detector uses three scalar features. The first is the acceleration magnitude

$$A = \|\mathbf{a}\|_2 = \sqrt{a_x^2 + a_y^2 + a_z^2}, \tag{3}$$

where a_x, a_y and a_z are the components sampled at the same instant and divided by g_0. Thus $A = 1$ means one g, and the thresholds 2.0 and 0.35 used below mean $2g$ and $0.35g$. In the firmware the raw signed counts are divided by 2048 counts per g, which is the scale for the $\pm 16g$ range, and a new sample arrives every 10 ms.

The second feature is the magnitude of the angular rate,

$$\Omega = \|\boldsymbol{\omega}\|_2 = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}, \tag{4}$$

where ω_x, ω_y and ω_z are the bias-corrected rates about the three sensor axes, in degrees per second. The raw values are divided by 16.4 counts per degree per second (the $\pm 2000^\circ/\mathrm{s}$ range) before the calibration means are subtracted. Taking the norm makes the gate indifferent to the axis along which the dominant rotation happens, and to its sign. The detector compares Ω with $90^\circ/\mathrm{s}$ when looking for a rotation and with $30^\circ/\mathrm{s}$ when checking for rest; these are thresholds and not part of the formula.

The third feature is the tilt, the angle between the present acceleration vector and the reference,

$$\Theta = \frac{180}{\pi} \arccos\left[\mathrm{clip}\left(\frac{\mathbf{a}^{\mathsf{T}}\mathbf{r}}{\|\mathbf{a}\|_2 \|\mathbf{r}\|_2}, -1, 1\right)\right], \tag{5}$$

where $\mathbf{a}^{\mathsf{T}}\mathbf{r} = a_x r_x + a_y r_y + a_z r_z$ is the dot product, $\mathbf{r}$ is the mean of the 200 accepted calibration samples, and $\mathrm{clip}(u, -1, 1) = \min(1, \max(-1, u))$. Dividing by the two norms gives the cosine of the angle, the clipping protects arccos against round-off just outside its domain, and the factor $180/\pi$ turns radians into degrees, so that $\Theta \in [0, 180]$. If the product of the norms is below 10^{-6} the implementation returns a tilt of zero. Since Θ is computed from the instantaneous acceleration, it measures the lean of the vehicle properly only when the motion is small, that is, during the rest after a fall, which is where we use it.

With these features, the decision proceeds as follows. Riding activity is present when $|A - 1| \geq 0.08$ or $\Omega \geq 4^\circ/\mathrm{s}$. Evidence of activity accumulates up to 1000 ms and decays at half that rate when there is no activity, and while the evidence is full a 5000 ms deadline for active riding is refreshed. Once riding is qualified, a candidate starts when $A \geq 2.0$ or $A \leq 0.35$. A rotation with $\Omega \geq 90^\circ/\mathrm{s}$ must occur within 750 ms of the candidate. After that the detector waits for the rest condition

$$R = (\Theta \geq 55^\circ) \wedge (0.70 \leq A \leq 1.30) \wedge (\Omega \leq 30^\circ/\mathrm{s}), \tag{6}$$

where $\wedge$ is logical AND, so R is true only when the motorcycle is strongly tilted, the acceleration is close to $1g$ (within $\pm 0.30g$), and the vehicle is hardly rotating. The comparisons include their limits. The predicate is instantaneous; the time requirements are imposed by the state machine, not by Equation (6).

A crash is confirmed when R has held for 2000 ms, and the verification is abandoned 6000 ms after the candidate. Confirmation starts the alarm and the 15,000 ms cancellation window. The state machine needs constant memory and constant work per sample.

3.7 Algorithm and flowchart

Table 3 gives the firmware procedure in the order in which it runs, from initialisation of the sensor to the single event submission. It also shows where a sensor or network failure changes the normal path.

Table 3: Algorithm 1: firmware procedure from sensor initialisation to the one-shot incident submission. Each row is one step of the implemented control path, with its timing or failure branch. Samples are taken every 10 ms (100 Hz) and calibration uses 200 samples. A failed sensor read skips the detector update. The deadline step records the outcome at the service and does not imply that a notification reached anyone.

Step	Operation
1	Configure the GPIO pins and the MPU-6050; stop safely if the sensor does not answer.
2	Collect 200 samples; if the device moved or the data are invalid, repeat until a valid reference is obtained.
3	Start Wi-Fi association in the background and set up the 10 ms sampling schedule.
4	Check for a press of the cancel button; cancel a running countdown, or acknowledge an alert and silence the outputs.
5	At each sampling deadline read the 14-byte sensor burst; on failure log a warning and skip the detector update.
6	Compute A, Ω and Θ; update the activity evidence and the candidate, rotation and rest states.
7	On confirmation set the LED high, start the 2400 Hz tone and record the start of the countdown.
8	When 15 s have elapsed, perform one event operation and record whether it was submitted, disabled, skipped for lack of Wi-Fi, or skipped for lack of cloud.
9	Keep the local warning on until the rider acknowledges it; continue sampling.

The flowchart in Figure 4 shows the same procedure as a sequence of decisions, with its two return paths: a rejected candidate returns to sampling, and a cancellation by the rider stops the warning before the event falls due.

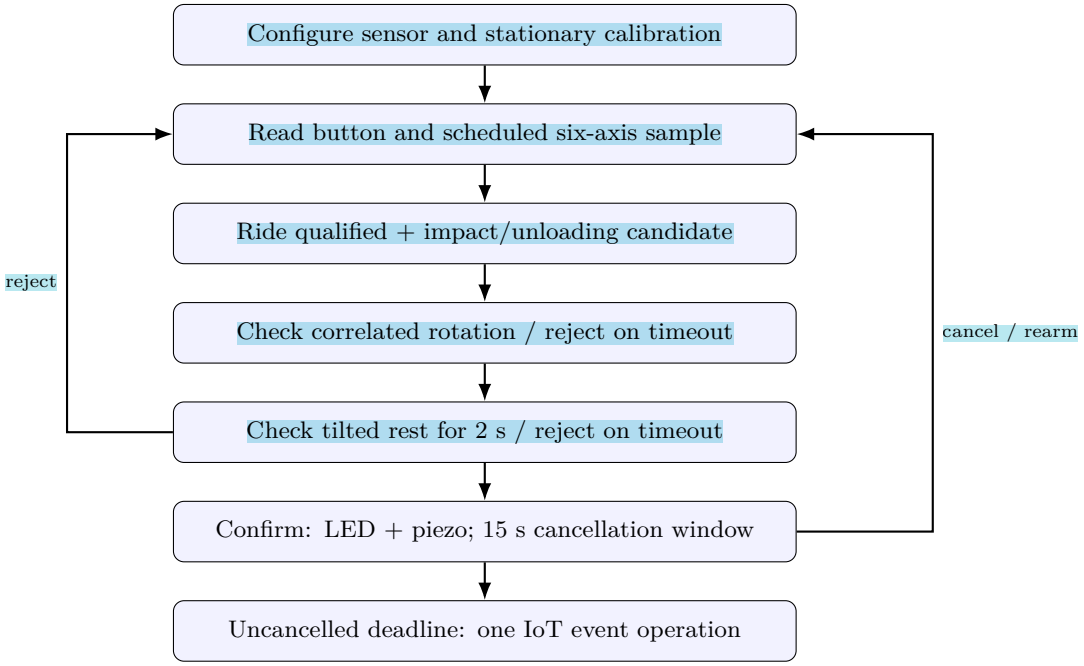


Figure 4: Flowchart of a successful detection, with the return paths for rejection and cancellation. A failed sensor read also returns to the sampling loop.

3.8 Baselines and metrics

We compare CrashGuard with three other rules. The *acceleration-only* rule fires as soon as $A \geq 2.0$. The *strict ordered* rule needs $A \geq 4.0$, then $\Omega \geq 200°/\text{s}$ within 500 ms, and then a tilt of at least 60° with $0.70 \leq A \leq 1.30$ held for three seconds, all within eight seconds of the impact. The third rule follows the staged structure of the motorcycle fall detector of Boubezoul et al. [1], and we refer to it by their name. It has three stages. A fall starts when the roll rate $|\omega_x|$ reaches $60°/\text{s}$, or when $A \geq 2.5$ or $A \leq 0.40$. The lean must then reach $\Theta \geq 55°$ within 2 s, otherwise the rule resets. Finally the rest condition $\Theta \geq 55°$, $0.70 \leq A \leq 1.30$, $\Omega \leq 30°/\text{s}$ must hold continuously for 1.5 s, and the attempt is abandoned 6 s after the start. To be clear, these thresholds are our own choices, written to follow the structure of the published method; we do not claim that they are the exact values of the original paper, so this rule is an approximation of their detector and not a copy. Unlike CrashGuard, it has no ride qualification.

Let TP, TN, FP and FN be the numbers of true positives, true negatives, false positives and false negatives. All of them are counts of whole events, and in every experiment below the denominators of the metrics are positive. The first metric is accuracy,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{7}$$

the fraction of all events which are classified correctly. The denominator is the size N of the evaluation set, $N = 675$ for the primary set and $N = 270$ for the fresh set. As there are many more negative events than positive ones, accuracy can be high even when many crashes are missed, which is why we never report it alone.

Sensitivity (also called recall, or the true-positive rate) is the fraction of crashes which are detected,

$$\text{Sensitivity} = \frac{TP}{TP + FN}. \tag{8}$$

The denominator is the number of labelled crashes, 150 in the primary set and 60 in the fresh set. This is the metric which shows missed crashes directly, and we also give it separately for low-side and high-side crashes. For example, the fresh CrashGuard result is $43/(43 + 17) = 0.717$, that is 71.7%.

Specificity is the fraction of non-crash events which are correctly rejected,

$$\text{Specificity} = \frac{TN}{TN + FP}, \tag{9}$$

with 525 negatives in the primary set and 210 in the fresh set. It describes nuisance alarms, but only within the negative events we chose to generate, so it has to be read together with how many and how varied these events are. For the fresh CrashGuard result $TN = 210$ and $FP = 0$, so the observed value is 1. This is a finite sample, and below we give a Wilson interval for it rather than claim perfect rejection.

Precision is the fraction of detections which are real crashes,

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{10}$$

The denominator counts all events predicted positive, whatever their true class, so precision depends on how common crashes are in the test set. If a rule predicted no positive event at all, the ratio would be undefined and our code would return zero; this never happens in the results reported here. For CrashGuard on the fresh set, precision is $43/(43 + 0) = 1$.

The F1-score combines precision and sensitivity,

$$F_1 = \frac{2TP}{2TP + FP + FN}, \tag{11}$$

which is the harmonic mean of the two written in terms of counts; the subscript 1 means that both are weighted equally. F1 is penalised by both kinds of error, and we report it as a decimal between 0 and 1, whereas accuracy, sensitivity and specificity are given in percent. For the fresh CrashGuard counts, $F_1 = 86/103 = 0.835$.

The false-positive rate is

$$\text{FPR} = \frac{FP}{TN + FP}, \tag{12}$$

the fraction of non-crash events which raise an alarm, which is one minus the specificity. For the acceleration-only rule on the fresh set, $FP = 108$ and $TN = 102$, so FPR $= 108/210 = 51.4\%$; for CrashGuard the numerator is zero. Note that this is a rate per event. A rate of false alarms per hour of riding would need the riding time as its denominator, which our data do not provide.

The delay of a detection is the time of confirmation minus the time at which the successful candidate started, and it is computed only for detected crashes. The acceleration-only rule has, by definition, no confirmation hold and thus zero delay. For a proportion $\hat{p} = x/n$ with x successes out of n events, we give 95% Wilson intervals [17],

$$\begin{aligned} p_{\mathrm{L}} &= \frac{\hat{p} + z^2/(2n) - z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \\ p_{\mathrm{U}} &= \frac{\hat{p} + z^2/(2n) + z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \end{aligned} \tag{13}$$

where p_{L} and p_{U} are the lower and upper limits of the interval, and $z = 1.959964$ is the 97.5th percentile of the standard normal distribution. For sensitivity $x = TP$ and $n = TP + FN$, and for specificity $x = TN$ and $n = TN + FP$. The constants 2 and 4 come from the Wilson score construction. Unlike the simple normal interval, the Wilson interval stays informative when all observed events are successes, as happens for our specificity. The intervals describe uncertainty due to the finite number of events; they say nothing about whether the generator resembles real riding.

Finally, to test whether two rules differ on the same events, we use McNemar's test with Edwards' continuity correction [18],

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \tag{14}$$

where b is the number of events which CrashGuard classifies correctly and the other rule does not, and c is the number of events for which the opposite holds. Events on which the two rules agree do not enter the statistic. Under the null hypothesis that both rules have the same error rate, χ^2 follows a chi-squared distribution with one degree of freedom. When there are few discordant events ($b + c < 25$) we use instead the exact two-sided binomial p-value, $p = 2\sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 0.5^{b+c}$.

4 Experimental Results and Discussion

4.1 Experimental conditions

The C++17 programs are compiled with optimisation and strict warnings. Python with NumPy, SciPy and Matplotlib generates the data, replays the detector, computes the metrics and draws the plots. Every event is processed by a freshly reset detector. The firmware harness uses a virtual clock, a 14-byte register buffer, instrumented GPIO and tone outputs, and a local Blynk-compatible sink. It runs the unchanged firmware and returns, on request, a failed probe, a truncated read, a lost link or a cloud timeout. No network credentials are needed.

We re-ran the pipeline on the 675 primary events and obtained the same confusion counts as before, and then added the 270 fresh events, so 945 events and 945,000 samples were processed in total. Besides these, we ran 14 firmware scenarios, 39 DC operating points and nine battery-discharge cases. These experiments test different layers of the system, and we report their counts separately.

4.2 Original and fresh performance comparison

Table 4 compares the four rules on the 675 primary events, and its confusion counts show where each score comes from. CrashGuard rejects far more non-crash events than the acceleration-only rule and the Boubezoul rule, while the strict ordered rule, which also rejects every negative event, loses many more crashes.

Table 4: Event-level comparison of four rules on the same 675 primary events (seed 20260828; 150 crashes and 525 non-crash events). TP, TN, FP and FN are event counts. Acc., Sens. and Spec. are accuracy, sensitivity and specificity in percent, and F1 lies in $[0, 1]$. Since every row sees identical input, the differences come from the rules alone.

Detector	TP	TN	FP	FN	Acc. (%)	Sens. (%)	Spec. (%)	F1
Acceleration-only	134	236	289	16	54.8	89.3	45.0	0.468
Strict ordered	39	525	0	111	83.6	26.0	100.0	0.413
Boubezoul et al. (2013)	135	436	89	15	84.6	90.0	83.0	0.722
CrashGuard (Ours)	114	525	0	36	94.7	76.0	100.0	0.864

Table 5 repeats the comparison, with the rules unchanged, on the 270 fresh events. The question is whether the pattern seen on the primary set survives on a new random stream; no threshold was chosen by looking at these results.

Table 5: The unchanged rules on 270 fresh events (seed 20260911; 60 crashes and 210 non-crash events). Columns as in Table 4. This tests repeatability within the same generator; it is not a validation on other vehicles or road sessions.

Detector	TP	TN	FP	FN	Acc. (%)	Sens. (%)	Spec. (%)	F1
Acceleration-only	52	102	108	8	57.0	86.7	48.6	0.473
Strict ordered-threshold	12	210	0	48	82.2	20.0	100.0	0.333
Boubezoul et al. (2013)	52	176	34	8	84.4	86.7	83.8	0.712
CrashGuard (Ours)	43	210	0	17	93.7	71.7	100.0	0.835

On the primary set CrashGuard removes all 289 false alarms of the acceleration-only rule, and misses 20 more crashes than it. Against the Boubezoul rule, CrashGuard removes 89 false alarms (specificity 100.0% against 83.0%) and misses 21 more crashes (sensitivity 76.0% against 90.0%). The same pattern appears on the fresh set: CrashGuard has none of the 34 false alarms of the Boubezoul rule, which raises specificity from 83.8% to 100.0% and accuracy from 84.4% to 93.7% (F1 0.835 against 0.712), but it misses 9 more low-side crashes (71.7% against 86.7%). Neither rule, then, dominates the other. CrashGuard pays for its silence on non-crash events with missed slow crashes, and whether that is the right trade depends on how riders react to repeated false alarms, a question our data cannot answer.

McNemar's test confirms that CrashGuard and the Boubezoul rule make significantly different errors: $\chi^2 = 40.81$ ($p = 1.68 \times 10^{-10}$) on the primary set and $\chi^2 = 13.40$ ($p = 0.00025$) on the fresh set. The test should, however, be read with care. On the fresh set the discordant events are 34 non-crash events which only the Boubezoul rule gets wrong and 9 crashes which only CrashGuard misses, so the significance reflects mainly the false alarms of the Boubezoul rule; it is not evidence that CrashGuard is better at detecting crashes. Against the strict ordered rule, CrashGuard detects significantly more crashes without any extra false alarm ($\chi^2 = 73.01$, $p < 10^{-15}$ on the primary set; $\chi^2 = 29.03$, $p = 7.1 \times 10^{-8}$ on the fresh set). Against the acceleration-only rule the difference is larger still ($\chi^2 = 232.44$, $p < 10^{-15}$ on the primary set; $\chi^2 = 82.09$, $p < 10^{-15}$ on the fresh set).

On the fresh set the 95% Wilson interval for sensitivity is 59.2–81.5% and the observed specificity of 100.0% has the interval 98.2–100.0%. On the primary set the intervals are 68.6–82.1% for sensitivity and 99.3–100.0% for specificity. Observing no false alarms in a finite set of negatives does not imply that none would occur under every riding condition.

Table 6 gives the CrashGuard counts for each subset of the data. It keeps apart the retrospective development, validation and test breakdown of the primary set and the fresh-seed experiment, which play different roles.

Table 6: CrashGuard confusion counts and F1 for each whole-event subset. The primary simulation is split within each of the nine scenarios into 45 development, 15 validation and 15 test events (405/135/135 in total). The fresh-seed set has 30 further events per scenario. TP, TN, FP and FN add up to the number of events. No weights are fitted in any subset, since the rules are fixed.

Partition	Events	TP	TN	FP	FN	F1
Development	405	67	315	0	23	0.854
Validation	135	23	105	0	7	0.868
Retrospective test	135	24	105	0	6	0.889
Fresh-seed test	270	43	210	0	17	0.835

The F1 values of the three primary subsets lie between 0.854 and 0.889, and the fresh value, 0.835, is a little below them. Differences of this size are what one would expect from subsets with 30 positives or fewer.

4.3 Scenario analysis and graphs

All seven negative classes are rejected completely in both sets, and no high-side crash is missed: 75 of 75 in the primary set and 30 of 30 in the fresh set are detected. For low-side crashes the counts are 39 of 75 and 13 of 30. Hence every one of the 36 primary misses and the 17 fresh misses is a low-side crash. The low-side profiles have weaker impacts, slower motion and a smaller final lean than the high-side ones, and the fixed gates accept only part of that distribution.

Figure 5 shows the fraction of correct decisions per class for both sets. Its message is simple: overall accuracy conceals one class that fails often, while the other eight are handled without error in these finite sets.

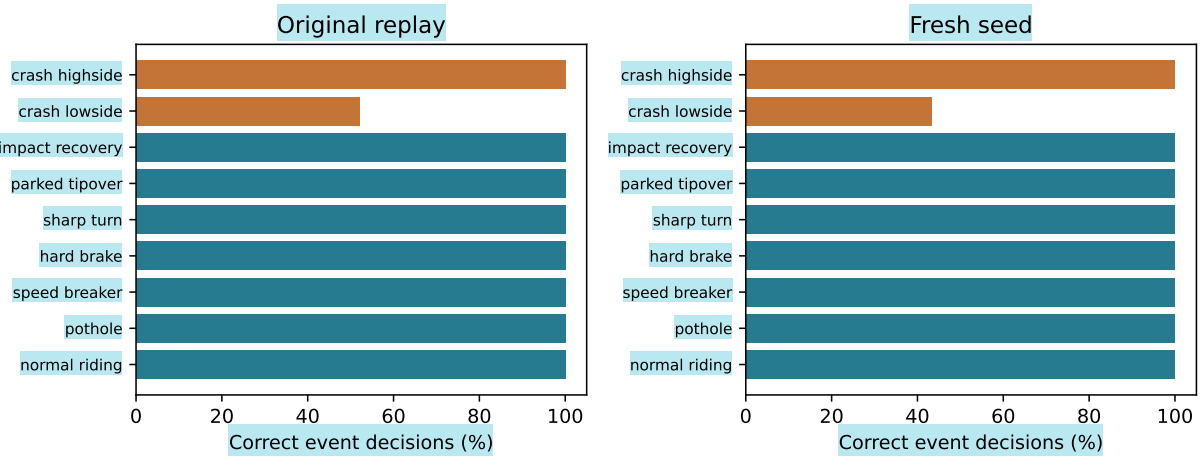


Figure 5: Fraction of correct event decisions per scenario in the primary replay (left; 75 events per class) and in the fresh-seed set (right; 30 events per class). Orange bars are crash classes and blue bars non-crash classes.

Figure 6 shows the three features over time for one generated pothole, one parked tip-over and one low-side crash. The pothole produces a sharp acceleration peak above $3g$, but its rotation stays near the $90°/s$ line and the tilt never rises. The parked tip-over and the crash both rotate fast and end strongly tilted. Their difference lies in the ride qualification, since the tip-over happens without prior riding. This is why the same peak value need not lead to the same decision.

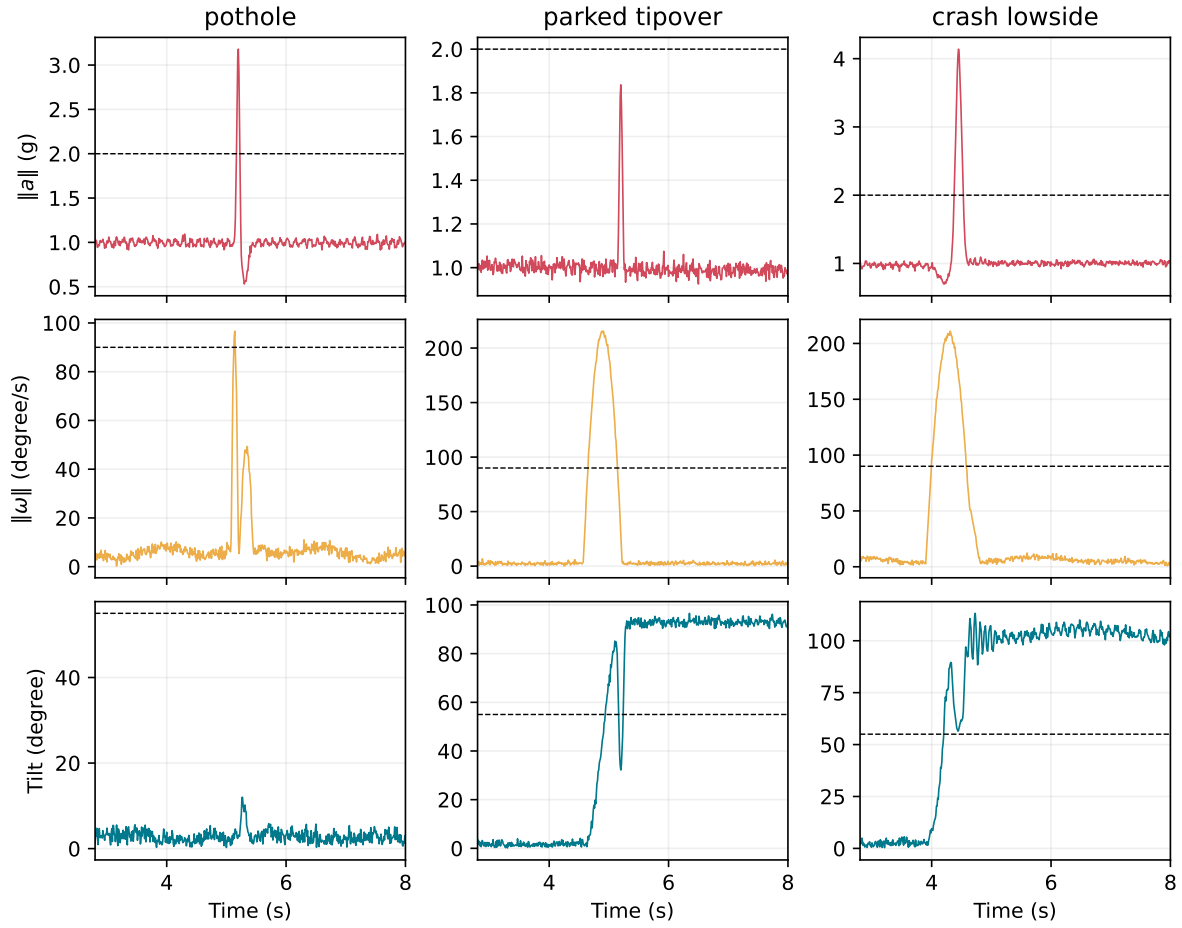


Figure 6: Acceleration magnitude $\|\mathbf{a}\|$ (top), angular-rate magnitude $\|\boldsymbol{\omega}\|$ (middle) and tilt (bottom) for a generated pothole, parked tip-over and low-side crash. Dashed lines mark the $2g$, $90°/s$ and $55°$ thresholds.

Figure 7 puts the fresh confusion matrix next to the distribution of confirmation delays. The two panels have different denominators: the matrix counts all 270 events, the histogram only the 43 detected crashes. Most delays lie between 2.2 and 2.4 s, that is, just above the 2 s rest hold.

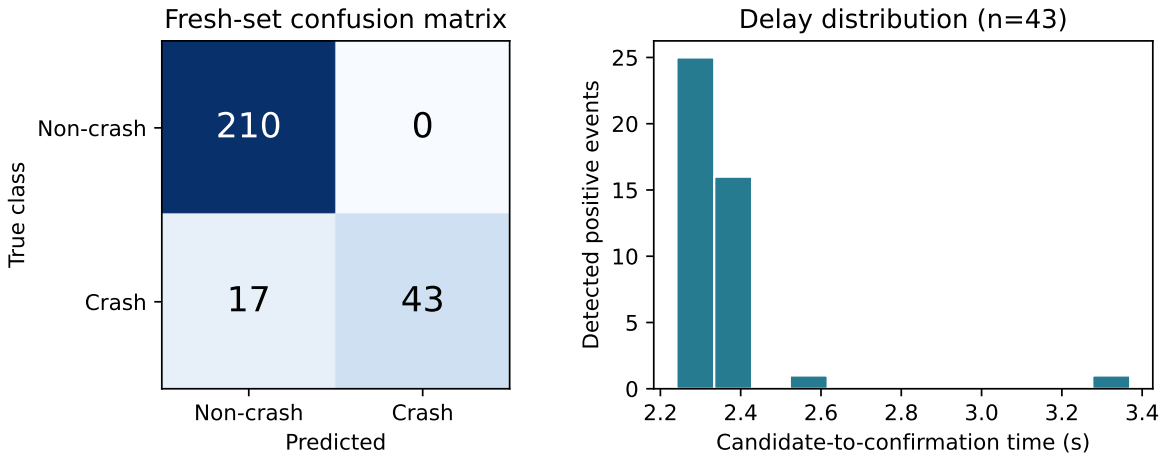


Figure 7: Fresh-set confusion matrix (left) and the candidate-to-confirmation delays of the 43 detected crashes (right). Missed crashes appear only in the matrix.

4.4 External public-data gate analysis

Table 7 summarises the four public fall windows after unit conversion and resampling. The peak and minimum acceleration tell us whether the unloading gate opens, and the peak angular rate whether the rotation gate opens. The last column records only that both gates are crossed somewhere in the window, not that the full decision would be made.

Table 7: Extreme values of the inertial signals in four public PTW fall windows, after anti-aliased resampling from 1 kHz to 100 Hz. Each row covers 3 s centred on the event. Peak and minimum A are in g; peak Ω is in degrees per second. "Both gates" means that the acceleration condition ($A \geq 2.0$ or $A \leq 0.35$) and the rotation condition ($\Omega \geq 90°/s$) are each met somewhere in the window; it says nothing about their timing or about the rest condition.

Fall recording	Peak A (g)	Min. A (g)	Peak Ω (°/s)	Both gates
Slippery straight road	1.169	0.013	179.4	Yes
Leaning motorcycle	1.878	0.041	144.1	Yes
Roundabout	1.690	0.011	150.5	Yes
Curve	1.436	0.130	151.6	Yes

All four falls open both CrashGuard gates, and in each of them it is the unloading branch ($A \leq 0.35$) that opens, since no peak reaches $2g$. None of them reaches the $4g$ and 200°/s gates of the strict rule, which explains part of its poor recall. On the other hand, three of the seven manoeuvres (acceleration in a curve, acceleration on a straight line, and hard braking) also open both CrashGuard gates somewhere in their recordings. The gates alone, therefore, do not separate falls from manoeuvres; that job falls to the requirements on timing, riding and rest. We cannot check those requirements on these recordings, as the fall windows are short. The full features of all eleven recordings are in `external_ptw_features.csv`.

4.5 Timing, cancellation, and IoT communication

The median delay from candidate to confirmation is 2315 ms on the primary set and 2330 ms on the fresh set. The strict ordered rule needs 3120 and 3130 ms, and the Boubezoul rule 2280 and 2325 ms. Most of the delay is thus accounted for by the rest hold itself. With the intentional 15 s cancellation window added, the time from the candidate to the event deadline is 17.33 s at the fresh median, before any delay in connecting to the service.

In the harness, riding is qualified first, then a rotation is injected at 1500 ms and an impact at 1600 ms. The tilted rest starts at 1610 ms, confirmation follows at 3610 ms, and the event falls due at 18,610 ms. The test checks that the state is still `COUNTDOWN` at 18,600 ms and becomes `ALERT_DUE` ten milliseconds later. After confirmation GPIO26 is high and the tone is 2400 Hz. The simulated service receives exactly one `crash_confirmed` event, whose description is 143 bytes long.

In a separate test the cancel button on GPIO27 is pressed at 4010 ms. The outputs go off, and the detector stays idle until the end of the run at 20,000 ms. The event path distinguishes four outcomes: submission to the service, a disabled configuration, a Wi-Fi failure and a cloud failure. The event description gives the acceleration, angular rate and tilt at confirmation, and states that no location fix is available, since the device has no GNSS receiver. The event identity and payload follow the Blynk firmware event interface [14].

The virtual-hardware image together with the state traces works as a replay dashboard, in which the sensor values, the countdown, the alarm outputs and the event outcome all belong to one run. A deployed application could show the same fields through a Blynk device template. Continuous telemetry on virtual pins would be an extension; in the sketch we evaluate, the only remote operation is the event submission.

4.6 Threshold sensitivity and calibration analysis

To see how much the results depend on the chosen thresholds, we varied one parameter at a time, kept the others at their configured values, and replayed the 675 primary events for each setting. The parameters and ranges were the following.

1. The rest tilt threshold Θ_{tilt}, 13 values from 40° to 70° in steps of 2.5°.
2. The rotation gate Ω_{rot}, 10 values from 60 to 150°/s in steps of 10°/s.
3. The rest hold T_{hold}, 6 values from 1.0 to 3.5 s in steps of 0.5 s.

This gives 29 settings in all. Because the sweep reuses the events on which the configuration was developed, it describes how the rules behave and is not an independent test. The results are shown in Figure 8.

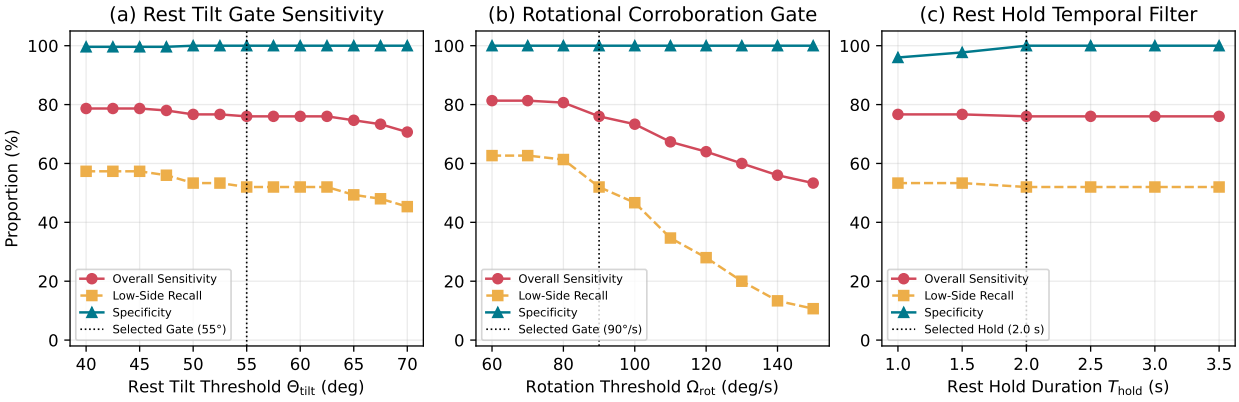


Figure 8: One-at-a-time threshold sweeps on the 675 primary events: (a) rest tilt threshold, (b) rotation gate, (c) rest hold. Each panel shows overall sensitivity, low-side recall and specificity; the dotted line marks the configured value (55°, 90°/s, 2.0 s).

Tilt. Specificity stays at 100% for every tilt threshold down to 50°. From 47.5° downwards two non-crash events are accepted (99.6%). Within the range without false alarms, the tilt threshold matters surprisingly little: sensitivity is 76.7% at 50–52.5° and 76.0% from 55° to 62.5°, a difference of one event. Going down to 45° recovers four more low-side crashes (low-side recall 57.3%, overall 78.7%) at the price of the two false alarms. Above 62.5° sensitivity falls, and at 67.5° and 70° even high-side crashes begin to be lost (98.7% and 96.0%); at 70° overall sensitivity is 70.7%. We conclude that the tilt gate is not the main cause of the low-side misses.

Rotation. Specificity stays at 100% over the whole range from 60 to 150°/s, while low-side recall depends strongly on the gate. It is 62.7% at 60 and 70°/s, 52.0% at the configured 90°/s and only 10.7% at 150°/s. Thus on these data a rotation gate of 70°/s would detect eight more low-side crashes (overall sensitivity 81.3% instead of 76.0%) without a single false alarm. This is perhaps the most useful outcome of the sweep: the configured point is not the best one among those free of false alarms. We have not changed the configuration,

however, because the observation comes from the development data and the rejection of negatives may rest largely on the other conditions; a lower rotation gate needs to be checked on held-out events before it is adopted.

Rest hold. Here the picture is reversed. Sensitivity hardly moves (76.7% at 1.0 and 1.5 s, 76.0% from 2.0 s), but specificity drops to 97.7% (12 false alarms) at 1.5 s and to 96.0% (21 false alarms) at 1.0 s. The sweep does not record which classes these false alarms come from. A likely source is the impact-with-recovery class, whose generated lean lasts between 0.7 and 2.1 s, but we have not verified this. Longer holds bring no gain in specificity, and since the hold is part of the confirmation delay, every extra half second of hold adds about half a second of delay by construction. So the 2 s hold is the shortest hold, among those tried, which removes all false alarms.

4.7 Firmware implementation verification and code defect discovery

Table 8 pairs each injected condition with the intended response and the trace actually observed. Most cases check that the firmware behaves as specified. Three of them, T9, T11 and T13, isolate defects in this particular code. We stress that these are defects of our prototype implementation and not properties of IoT hardware in general; the harness found them before any board was flashed.

Table 8: Fourteen experiments with the exact firmware and deterministic models of the sensor, GPIO, clock and service. ID names the case, Stimulus the injected input or failure, Expected the required behaviour, and Actual the observed trace with its interpretation. Times are virtual-clock milliseconds unless seconds are written. Alarm 1/0 means the local warning is commanded on/off. In T9, T11 and T13 a passing assertion reproduces a known weakness; it does not certify tolerance to faults.

ID	Stimulus	Expected behaviour / criterion	Actual result and interpretation
T1	Sensor absent at boot	Probe failure; safe stop; alarm off	Fatal marker and safe-stop loop observed; alarm 0. Conforms.
T2	Ten-byte sensor burst	Reject incomplete 14-byte read	Warning emitted; state remains IDLE. Conforms.
T3	I^2C error status	Skip invalid sample	Warning emitted; state remains IDLE. Conforms.
T4	100°/s during calibration	Reject motion above 5°/s RMS	Calibration rejected. Conforms.
T5	Stationary gravity input	Accept 200 valid samples	Calibration accepted after 200 samples. Conforms.
T6	ADC returns 1850 mV	Reconstruct cell voltage with factor two	3.700 V returned. Conforms.
T7	Cancel at 4010 ms	Silence outputs and suppress deadline	Alarm 0; IDLE through 20 s. Conforms.
T8	Uncancelled crash	15,000 ms from confirmation to one event	3610 ms confirmation; 18,610 ms deadline; one event. Conforms.
T9	Missing reads during rest	Desired: require continuous valid rest evidence	Confirmation at 3610 ms despite a sample gap. Continuity defect exposed.
T10	Wi-Fi unavailable at deadline	Explicit skip; local alarm kept	Zero sink events; Wi-Fi skip marker; alarm 1. Conforms.
T11	Cloud connection fails	Explicit failure; characterise blocking	Zero sink events; 5000 ms blocking; alarm 1. Blocking risk exposed.
T12	Connected simulated sink	Correct event code and bounded payload	One event; code `crash_confirmed`; 143 bytes. Conforms.
T13	Wi-Fi recovers after deadline	Desired: eventual incident delivery	Zero events after recovery. Missing retry exposed.
T14	Build without credentials	Disabled marker; local alarm kept	Not-configured marker; alarm 1. Conforms.

All fourteen assertions pass; for T9, T11 and T13, passing means that the known defect is reproduced. We describe the three defects in turn.

T9: rest-timer continuity. The rest timer measures elapsed time between timestamps and does not check that samples actually arrived in between. If the I^2C reads fail during the rest, the time still counts. In the test a valid rest sample at 1610 ms is followed by bus failures at 2000 and 3000 ms and by a valid sample at 3610 ms, and the crash is confirmed although the sensor was silent for 2000 ms. The fix is to restart the rest timer whenever the gap between valid samples exceeds, say, 50 ms.

T11: blocking connection. The cloud connection call is synchronous and has a 5000 ms timeout. When the service cannot be reached, the call stops the main loop for those 5 s and sampling deadlines are missed. Production firmware needs a non-blocking network state machine.

T13: no retry. If an incident falls due while Wi-Fi is down, it is skipped and the state moves to `ALERT_COMPLETED`. When Wi-Fi comes back, nothing is sent. The event has to be stored and retried.

The GPIO and tone assertions concern only the commanded outputs. A disconnected piezo or a dead LED cannot be detected by the present firmware, which has no feedback from its actuators. Checking that the outputs physically work would need additional sensing, and we leave it for future work.

4.8 Expected performance versus actual performance

Table 9 sets each design objective against the experiment that tests it and states what follows. It puts classification, timing, recovery and endurance side by side, including the objectives which were not met.

Table 9: Objectives, observations and interpretation. The classification rows give correct decisions or false alarms over the relevant number of events in the primary and fresh sets. The timing rows use the firmware's virtual clock, and the recovery rows say whether continuous rest evidence and eventual delivery are achieved. The endurance row refers to the 120 mA, 5 V, 90% efficiency case. Meeting one row does not imply that the others are met.

Objective	Actual experiment	Interpretation
Reject road disturbances	0/525 primary and 0/210 fresh false alarms	Achieved in both scenario sets.
Detect crash-like events	114/150 primary; 43/60 fresh	Low-side sensitivity is the main weakness.
Allow cancellation	Alarm silenced at 4010 ms; IDLE through 20 s	Cancellation conforms.
Enforce 15 s alert interval	18,610 − 3610 = 15,000 ms	Exact deadline on the virtual clock.
Keep valid rest evidence	Sensor gap spans the rest timer	Timer must restart after a gap.
Recover event delivery	Zero events after the link returns	Persistent retry needed.
Run for 12 h in the nominal case	12.64 h at 120 mA and 90% efficiency	Met in the nominal case; heavier loads fall short.

4.9 Power simulation

The battery is modelled as a reservoir of usable charge that is drawn down step by step; the operating time is an output of the model, not an input. The initial charge is

$$q_0 = C_{\mathrm{nom}} f_{\mathrm{use}} = (2.6\,\mathrm{Ah})(0.90) = 2.34\,\mathrm{Ah}, \tag{15}$$

where $C_{\mathrm{nom}} = 2.6$ Ah is the nominal capacity of the cell, $f_{\mathrm{use}} = 0.90$ is the fraction of it which we treat as usable, and the subscript 0 marks the start of the discharge. Both inputs are fixed parameters of the simulation.

During discharge the cell voltage falls linearly with the remaining charge,

$$V(q) = V_{\mathrm{min}} + (V_{\mathrm{max}} - V_{\mathrm{min}})\frac{q}{q_0} = 3.0\,\mathrm{V} + (1.2\,\mathrm{V})\frac{q}{q_0}, \tag{16}$$

where q is the remaining usable charge in Ah, with $0 \le q \le q_0$, so that q/q_0 is the state of charge. The voltage is $V_{\mathrm{min}} = 3.0$ V when the cell is empty and $V_{\mathrm{max}} = 4.2$ V when it is full. The model is deliberately simple; a real lithium-ion cell has a flatter curve in the middle.

The boost converter holds the output at a fixed voltage, so the current drawn from the cell grows as the cell voltage falls. At integration step k,

$$I_{\text{cell},k} = \frac{V_{\text{out}} I_L}{\eta V(q_k)}, \tag{17}$$

where q_k is the remaining charge, $V_{\text{out}} = 5.0$ V is the output voltage, I_L is the load current (90, 120 or 180 mA) and η is the converter efficiency (0.80, 0.90 or 0.95). The numerator is the output power in watts, and dividing it by the efficiency and the cell voltage gives the cell current in amperes. All nine combinations of load and efficiency were simulated.

The charge is then updated by a forward Euler step,

$$q_{k+1} = q_k - \frac{I_{\text{cell},k} \Delta t_s}{\kappa_t}, \qquad \Delta t_s = 1\,\text{s}, \quad \kappa_t = 3600\,\text{s}\,\text{h}^{-1}, \tag{18}$$

where Δt_s is the time step and κ_t converts ampere-seconds to ampere-hours. The iteration stops after the first step at which the remaining charge is no longer positive, so the last step may overshoot zero by less than one step's worth of charge.

For the nominal case, $I_L = 0.120$ A and $\eta = 0.90$, the cell is empty after 45,490 steps of one second, that is after 12.64 h. Over the nine cases the runtime ranges from 7.49 to 17.78 h. Figure 9 shows these runtimes; each point is one complete integration and each curve keeps the efficiency fixed while the load varies.

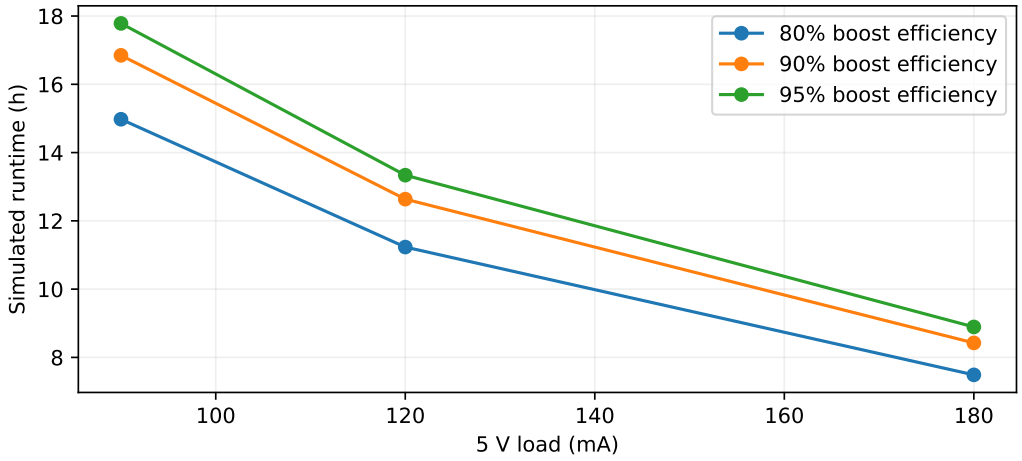


Figure 9: Simulated runtime for the nine combinations of 5 V load (90, 120, 180 mA) and boost efficiency (80, 90, 95%). The model includes the falling cell voltage and the converter loss.

The numerical model can be checked against a closed-form integral. For the same linear voltage profile and constant load and efficiency, the runtime in hours is

$$t_{\text{lin}} = \frac{\eta q_0 (V_{\min} + V_{\max})}{2 V_{\text{out}} I_L}, \tag{19}$$

where all symbols are as above, and $(V_{\min} + V_{\max})/2 = 3.6$ V is the mean cell voltage. The units reduce from Ah·V/(V·A) to hours. For the nominal case the formula gives 12.636 h, which agrees with the numerical result to within the one-second step.

For comparison, if the cell voltage is held constant at $V_{\text{const}} = 3.7$ V, with the other values unchanged, the runtime $t_{\text{const}} = \eta q_0 V_{\text{const}} / (V_{\text{out}} I_L)$ is 12.99 h. The difference of 0.35 h comes only from using 3.7 V in place of the mean of 3.6 V. Neither calculation includes ageing of the cell, temperature, or the voltage sag under short bursts of load such as Wi-Fi transmission.

4.10 Performance comparison with research literature

Table 10 is divided into two parts. Part A gives the four rules on the same 270 fresh events, so its rows can be compared directly. Part B gives published results from studies of human falls. Those studies use other

sensor positions (the shin, the waist), other motions and sometimes other goals, such as triggering an airbag before impact. Their numbers give context, but they cannot be ranked against Part A.

Table 10: Comparison by domain. Part A: the four rules on the identical 270 fresh simulated PTW events (seed 20260911; 60 crashes, 210 non-crash events). Part B: published results from studies of human motion with other protocols. Numbers may be compared only within Part A.

Method	Evaluation domain	Reported outcome	Relationship
Part A: motorcycle fall detection on identical fresh simulated events ($N = 270$)			
CrashGuard (Ours)	Fresh simulated PTW	Sens. 71.7%; Spec. 100.0%; Acc. 93.7%; 0 FP; F1 0.835	Staged state machine with ride qualification and 15 s cancellation.
Boubezoul et al. [1]	Same fresh events	Sens. 86.7%; Spec. 83.8%; Acc. 84.4%; 34 FP; F1 0.712	Rule following the published PTW algorithm; higher recall, lower specificity ($\chi^2 = 13.40$, $p = 0.00025$).
Acceleration-only	Same fresh events	Sens. 86.7%; Spec. 48.6%; Acc. 57.0%; 108 FP; F1 0.473	High recall, many alarms on road shocks ($\chi^2 = 82.09$, $p < 10^{-15}$).
Strict ordered	Same fresh events	Sens. 20.0%; Spec. 100.0%; Acc. 82.2%; 0 FP; F1 0.333	No false alarms, but misses 80% of crashes ($\chi^2 = 29.03$, $p = 7.1 \times 10^{-8}$).
Part B: literature from other domains (other sensors, mountings and objectives)			
Tian et al. (2026) [11]	Shin-mounted, human falls	Test sens. 90.5%; Acc. 93.1%; F1 0.916	Attention transformer for wearable fall detection; different mounting and mechanics.
KFall deep model [8]	Falls of older adults	Sens. 99.32%; Spec. 99.01%	Pre-impact detection on waist-worn data; different timing and dynamics.

Part A shows the trade-off which runs through this paper. CrashGuard gives up sensitivity on low-energy falls (71.7% against 86.7% for the Boubezoul rule) in return for no false alarms at all (specificity 100.0% against 83.8%). It seems plausible that riders would switch off a device which often raises false alarms, and in that case the trade is worth making; but we have no data on rider behaviour, and the question should be settled by a field study rather than by argument.

5 Limitations and Future Work

The main weakness of the detector is its low-side recall. CrashGuard missed 17 of the 30 fresh low-side crashes (43.3% recall), and these account for all its missed crashes in both sets. In a low-side crash the motorcycle slides down without a large impact, and the fixed gates do not see the slower ones. The sweep in Figure 8 suggests where to start: a lower rotation gate recovered eight low-side crashes in the development data without false alarms. This has to be confirmed on held-out events. Beyond that, one could track the lean angle adaptively, use the vehicle speed from wheel sensors or GNSS, or train a small sequence model for slow slides.

The generator uses proper rigid-body kinematics (Equation (2)), but much of the physics of a real motorcycle is missing from it: the travel of the front suspension, the flexibility of the frame, the nonlinear friction between tyre and road, engine vibration, and resonance of the mounting bracket. The detector must eventually be tested on instrumented crash tests and on long road recordings, over different surfaces and mounting positions. Note also that the low-side generator itself places some final roll angles below the 55° gate, so part of the low-side miss rate is a property of our data and part a property of the detector, and we have not separated the two.

The comparison with Boubezoul et al. depends on our own choice of thresholds for their staged method (Section 3.8). A different and equally reasonable choice could move their sensitivity and specificity, and so the conclusions of Table 5 about that rule should be read with this in mind.

The firmware tests revealed three defects which must be fixed.

1. *Rest-timer continuity (T9).* The rest timer must restart whenever the gap between valid samples exceeds a freshness bound, say 50 ms.
2. *Blocking network call (T11).* The synchronous 5000 ms connection call must be replaced by a non-blocking network state machine, so that sampling is not delayed.
3. *Event persistence (T13).* An incident which falls due during a network outage must be stored in non-volatile memory and retried, with back-off and deduplication, when the link returns.

Two further limitations are more basic: the device cannot tell the emergency services where the rider is, and it has never been built. A low-power GNSS receiver, or a Bluetooth link to the rider's phone, would supply the location. A real board, tested on a vibration bench and sealed against water and dust (IP67, say), is needed before anything can be said about the device outside simulation.

6 Conclusion

We have described CrashGuard, a crash detection and emergency alert prototype for powered two-wheelers, and evaluated it by simulation. A 100 Hz six-axis state machine decides whether a crash has occurred; a light and a buzzer warn the rider, who then has 15 seconds to cancel before an event is sent to the cloud.

On 270 fresh simulated events CrashGuard detected all 30 high-side crashes and rejected all 210 non-crash events, but it missed 17 of the 30 low-side crashes. Its overall sensitivity is therefore 71.7%, with 100.0% specificity, 93.7% accuracy and F1 = 0.835. On the 675 primary events the corresponding values are 76.0% sensitivity, 100.0% specificity and F1 = 0.864.

A rule following the staged method of Boubezoul et al. (2013) detected more crashes on the same fresh events (86.7%) but raised 34 false alarms (83.8% specificity; F1 0.712), and McNemar's test shows that the two rules make significantly different errors ($\chi^2 = 13.40$, $p = 0.00025$). The one-at-a-time sweeps show that the 2 s rest hold is what removes the false alarms, that the tilt threshold has little effect within the range without false alarms, and that a rotation gate of 70°/s instead of 90°/s would detect eight more low-side crashes on the development data without false alarms. This last point is the obvious next experiment.

Running the unchanged firmware against simulated peripherals proved useful in its own right: it found three defects before any hardware existed, namely the rest timer running across missing reads, a blocking cloud connection, and the missing retry after a network outage. The battery model gives 12.64 hours at the nominal 120 mA, 5 V load.

We have tried to report the weaknesses of the system as plainly as its strengths, and to make every number in the paper reproducible from the released code. We hope that this makes CrashGuard a useful starting point for others who build low-cost safety devices for two-wheeler riders.

CRediT authorship contribution statement

- **Adheesh Garg**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Ayan Gatani**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Aastha Gupta**: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft.
- **Syamasudha Veeragandham**: Supervision, Writing – review and editing.

Data Availability

All material is available on GitHub at `https://github.com/qwerty-dvorak/crashguard-iot/tree/main/paper`. The directory contains the manuscript in PDF and LaTeX, the bibliography, the ESP32 firmware,

the circuit definitions, the simulation and evaluation scripts, the generated datasets, the per-event predictions, the data partitions, the experiment logs, the figures and the results in machine-readable form, and all paths and commands resolve within it. The third-party PTW recordings are due to Boubezoul et al. [2]; the source URL and the SHA-256 checksum of their supplementary archive are given in the data documentation. Running `sh reproduce.sh` regenerates the motion data, replays the detectors, runs the firmware and power simulations, and rebuilds the manuscript. The README of the repository describes the procedure.

References

[1] A. Boubezoul, S. Espié, B. Larnaudie, and S. Bouaziz, "A simple fall detection algorithm for powered two wheelers," *Control Engineering Practice*, vol. 21, no. 3, pp. 286–297, 2013.

[2] A. Boubezoul, F. Dufour, S. Bouaziz, B. Larnaudie, and S. Espié, "Dataset on powered two wheelers fall and critical events detection," *Data in Brief*, vol. 23, p. 103828, 2019.

[3] S. Gelmini, S. C. Strada, M. Tanelli, S. M. Savaresi, and C. D. Tommasi, "A novel crash detection algorithm for two-wheeled vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 1, pp. 88–99, 2021.

[4] A. Sucerquia, J. López, and J. Vargas-Bonilla, "Sisfall: A fall and movement dataset," *Sensors*, 2017.

[5] E. Casilari, J.-A. Santoyo-Ramón, and J.-M. Cano-García, "Analysis of public datasets for wearable fall detection systems," *Sensors*, 2017.

[6] E. Casilari, R. Lora-Rivera, and F. García-Lagos, "A study on the application of convolutional neural networks to fall detection evaluated with multiple public datasets," *Sensors*, 2020.

[7] M. Musci, D. D. Martini, N. Blago, T. Facchinetti, and M. Piastra, "Online fall detection using recurrent neural networks," 2018. [Online]. Available: https://arxiv.org/abs/1804.04976

[8] X. Yu, J. Jang, and S. Xiong, "A large-scale open motion dataset (kfall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors," *Frontiers in Aging Neuroscience*, 2021.

[9] P. Rodegast, S. Maier, J. Kneifl, and J. Fehr, "On using machine learning algorithms for motorcycle collision detection," 2024. [Online]. Available: https://arxiv.org/abs/2403.09491

[10] J. Leoni, S. Gelmini, G. Panzani, M. Tanelli, and S. M. Savaresi, "Automatic ecall in powered two-wheelers: A dynamics-based approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, pp. 4365–4379, 2025. [Online]. Available: https://doi.org/10.1109/tits.2025.3545907

[11] J. Tian, S. Xu, P. Gerstoft, M. Sarkar, A. A. Moore, P. Mercier, and C. Paolini, "A multi-head attention transformer model for wearable in situ fall detection," *IEEE Access*, vol. 14, pp. 85 242–85 262, 2026.

[12] A. Pradhan, S. Alamgeer, R. Suvvari, S. T. Haque, and A. H. H. Ngu, "Dual-stream transformer with kalman-based sensor fusion for wearable fall detection," *Big Data and Cognitive Computing*, vol. 10, p. 90, 2026. [Online]. Available: https://doi.org/10.3390/bdcc10030090

[13] Blynk, "Device template documentation," 2026, accessed 11 September 2026. [Online]. Available: https://docs.blynk.io/en/concepts/device-template

[14] ——, "Log event: Firmware api," 2026, accessed 11 September 2026. [Online]. Available: https://docs.blynk.io/en/blynk-library-firmware-api/log-event

[15] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet.pdf

[16] CrashGuard project, "Shared detector, firmware, dataset generator and firmware experiment harness," 2026, reproducible source distributed with this report; see Data Availability and the repository README.

[17] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[18] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.